

**factor\_product(A, B)**

combines two factors (i.e.  $A$  and  $B$ ) over the union of their variables. The main issue is aligning entries that correspond to the same variable assignment; the provided assignment/index conversion functions are used for this (i.e. `assignment_to_index()`), after which the corresponding values are multiplied element-wise.

**factor\_marginalize(factor, var)**

removes selected variables by summing over their possible values. Each original assignment is reduced to the remaining variables, mapped to an output entry, and accumulated there. For example,

$$\phi_{X_3} = \sum_{X_2} \phi(X_2, X_3)$$

**observe\_evidence(factors, evidence)**

conditions the factors by setting assignments inconsistent with the observations to zero. The input factors are deep-copied first so later inference procedures do not modify the caller's data.

**compute\_joint\_distribution(factors)**

reconstructs the complete distribution by repeatedly multiplying the local factors according to:

$$P(X_1, \dots, X_n) = \prod_i P(X_i | \text{Parent}(X_i))$$

**compute\_marginals\_naive(V, factors, evidence)**

computes the marginal distribution of a single query variable  $V$  by working with the full joint distribution. The procedure is: construct full joint  $\rightarrow$  apply evidence  $\rightarrow$  marginalize all variables except  $V \rightarrow$  normalize.

If the normalization constant is zero, the evidence has zero probability and the conditional marginal is undefined, so a `ValueError` is raised.

**compute\_marginals\_bp(V, factors, evidence)**

avoids explicitly constructing the full joint distribution by passing local messages through the tree. For a message from node  $i$  to node  $j$ :

$$m_{i \rightarrow j}(X_j) = \sum_{X_i} \phi_i(X_i) \phi_{ij}(X_i, X_j) \prod_{k \in N(i) \setminus j} m_{k \rightarrow i}(X_i)$$

Thus node  $i$  combines its unary factor, all incoming messages from neighbours except  $j$ , and the pairwise factor between  $i$  and  $j$ , then marginalizes out  $X_i$ .

Because the graph is a tree, message passing is performed in two phases: an inward pass from the leaves to the root, followed by an outward pass from the root. Once both passes are complete, a node's marginal is obtained from its unary factor and all incoming messages, followed by normalization.

**factor\_sum() and factor\_max\_marginalize()**

For the MAP part, I reused almost the same message-passing structure as sum-product, but worked in log-space to avoid underflow as suggested. This makes the changes quite direct: multiplication becomes addition because

$$\log(ab) = \log a + \log b$$

while summing out a variable is replaced by taking the maximum. These are implemented using `factor_sum()` and `factor_max_marginalize()` respectively.

`factor_max_marginalize()` is therefore very similar to the earlier marginalization function. The difference is that when several assignments map to the same output entry, only the largest one is kept instead of adding them. The assignment that produced this maximum is stored in `val_argmax`, since this information is needed later to recover the MAP configuration.

**map\_eliminate(factors, evidence)**

first applies the evidence and converts the copied factors to log-space. Then an inward max-sum pass from the leaves to the root is run. Each message can be thought of as telling its parent: “for each value you could take, this is the best score achievable from my side of the tree.”

Once all messages reach the root, it is then combined with the root factor and the root state with the highest score is chosen. This gives the maximum log-probability, but not yet the complete assignment.

To recover the remaining variables, we backtrack down the tree. Given a selected parent value, look at the corresponding entry of the child-to-parent message and use its stored `val_argmax` to recover the child value that produced that maximum. Repeating this recursively gives the complete MAP assignment.

If the maximum log-probability is  $-\infty$ , the evidence has zero probability and `ValueError` is raised.